

Website Technical Architecture & Systems

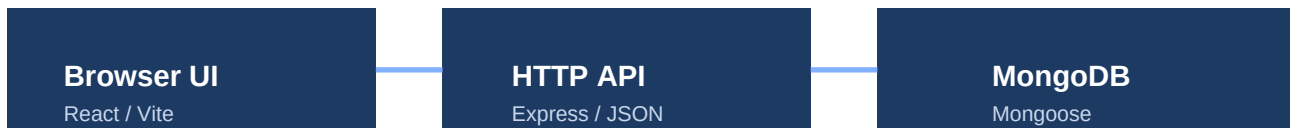
A practical map of the EI&TIC research-laboratory web application

FRONTEND

React 19 + TypeScript + Vite

API / SERVER

DATA FLOW



Pages, technologies, API wiring, data models, algorithms, authentication, uploads and setup.

Generated from the checked-out source tree. Secrets and .env values are intentionally excluded.

Contents

- 1. Executive overview
- 2. Technology inventory
- 3. Runtime architecture and request path
- 4. Frontend pages and user experience
- 5. Active API route catalog
- 6. Authentication, roles and access control
- 7. MongoDB entities and relationships
- 8. Algorithms and business workflows
- 9. File handling and publication lifecycle
- 10. Styling, UI libraries and assets
- 11. Configuration and local development
- 12. Build, lint and deployment notes
- 13. Source tree map and runtime caveats
- 14. Operational observations and verification checklist

1. Executive overview

This repository contains a research-laboratory website branded for EI&TIC. The public experience presents research areas, researchers, publications, news and collaboration information. Registered accounts can manage a researcher profile, submit publications or news, request access to private publication PDFs, and receive notifications. Administrators review submissions and manage users and content.

The implementation is a JavaScript/TypeScript MERN-style application: React renders the browser UI; Vite serves and builds the frontend; Express exposes the HTTP API; Mongoose defines MongoDB documents and queries. Authentication is cookie-based JWT. Uploaded publication PDFs and news images are stored on the server filesystem.

At a glance

- Frontend: React 19, TypeScript 6, React Router 7, Vite 8 and Tailwind CSS 4.
- Backend: Node.js ES modules, Express 5, Mongoose 8, MongoDB, JSON APIs and Multer disk uploads.
- Identity: local email/password with email verification; Google ID token verification; role-based admin/member/visitor access.
- Research workflows: publication approvals, profile publication summaries, private-PDF access requests, notifications and news approvals.

Runtime source of truth

Backend/server.js imports Backend/routes/auth.js and mounts it at /api/users. This combined router is the live endpoint source in the current server entry point. The separate auth.routes.js, publication.routes.js and user.routes.js files are present but not mounted by that entry point.

2. Technology inventory

Server-side runtime and packages

- Node.js with ECMAScript modules (package.json sets type=module); Express 5.1 for HTTP endpoints and middleware.
- MongoDB with Mongoose 8.16 for document schemas, indexes, model operations and references.
- bcryptjs hashes and checks local passwords; jsonwebtoken issues/verifies signed sessions; crypto creates opaque tokens, UUIDs, hashes and verification codes.
- google-auth-library validates Google Identity Services ID tokens against the configured OAuth client ID.
- Nodemailer sends email-verification and password-reset messages using SMTP configuration.
- Multer writes PDFs and images to disk; express-rate-limit throttles authentication, updates, access requests and uploads; Helmet applies common HTTP security headers.

Browser-side packages

- React 19 / React DOM 19 and TypeScript 6; Vite 8 with the React and Tailwind Vite plugins.
- React Router DOM 7 for client-side routes; Axios and Fetch for API calls.
- Tailwind CSS 4, shadcn-related theme CSS, tw-animate-css, Base UI React, class-variance-authority, clsx and tailwind-merge for styling and UI composition.
- Lucide React for icons; Motion for animation; Three.js for canvas / visual effects; Recharts for data visualization.
- xlsx (SheetJS) is imported by the admin UI for spreadsheet export; Geist Variable is packaged locally. Inter and Plus Jakarta Sans are also referenced via Google Fonts in the CSS.

Build and quality tools

- Frontend build runs TypeScript project build (tsc -b) before Vite production bundling. Oxcint is configured as the frontend linter.
- Root scripts: npm run dev starts Backend/server.js; npm run build delegates to the frontend build. Frontend additionally has dev, lint and preview scripts.

Versions above are the version ranges declared in the package manifests, not a claim that every runtime version was independently tested. The root package owns backend dependencies; Frontend/package.json owns browser dependencies.

3. Runtime architecture and request path

System boundary

During frontend development, the browser visits the Vite app (normally localhost:5173). Frontend/vite.config.ts proxies /api and /uploads to http://localhost:5000. The browser therefore calls relative paths such as /api/users/login and /api/users/publications without hardcoding the API host in each component.

```
Browser React page / component
-> Axios or Fetch: /api/users/... (relative URL)
-> Vite dev proxy: http://localhost:5000
-> Express middleware, active router and authorization
-> Mongoose model/query
-> MongoDB collection
<- JSON response, cookie, or streamed file
```

Server startup and middleware order

- Backend/server.js loads environment configuration, disables the x-powered-by header, enables Helmet and parses JSON request bodies up to 5 MB.
- It serves /uploads/actualites as static content only when the requested file extension is jpg, jpeg, png, webp or gif.
- A request logger prints method and path. /health returns a small JSON health response.
- Mutating /api/users requests with an Origin header are rejected when the origin differs from APP_URL; GET, HEAD and OPTIONS bypass this check. Requests without Origin are not rejected by this middleware.
- The app mounts Backend/routes/auth.js at /api/users. Before listening, it connects to MongoDB, ensures actualites collection existence, and warns when legacy publication collections are present.

Payload and response formats

- Most API operations exchange JSON. Publication and news submissions use multipart/form-data so a file can accompany metadata.
- The API uses MongoDB ObjectId values in resource URLs and responses. Public publication listing removes filesystem paths and private access-user IDs from the response.
- The PDF endpoint streams the file, while news images are referenced through /uploads/actualites/... URLs.

4. Frontend pages and user experience

Client-side routes (case-sensitive paths as coded)

- / and unknown paths redirect to /Accueil. The navigation bar links to /Accueil, /Publications, /Chercheurs, /Actualites and /Collaboration.
- /Accueil: landing/home page with lab highlights, research content, statistics, news and calls to action. Some stats and featured content are local demo data; researchers, publications and news are also requested from the API.
- /AxesdeRecherche: research-axis overview. The current page includes static example axes.
- /Chercheurs: public researcher directory, researcher cards, profile details and navigation to a profile.
- /Publications: publication list, search/filter/sort, publication cards, PDF modal, private-access status/request, and member/admin submission form.
- /Actualites: news listing/detail experience and a submission form for eligible accounts; fallback article data exists in the page.
- /Collaboration: collaboration information page, currently static content.
- /Profile: signed-in user's profile, or a public profile when researcherId is supplied in the query string. ProfileExperience contains sections for education, work history, skills, languages, links, research and publications.
- /AdminDashboard: admin-only dashboard with statistics, user/content management, pending-review queues and publication/news editing.
- /SignIn and /SignUp: local and Google account flows. /ForgotPassword, /ResetPassword and /VerifyEmail cover account recovery and verification.

Application state and presentation

App.tsx holds the current user in React state, fetches `/api/users/me` on load and redirects unauthenticated admin access to sign-in. Role 0 users can enter the admin dashboard. Navbar presents menus, session logout, notifications and polling. The UI is localized largely in French; route names and code identifiers use mixed French/English.

PublicationsView applies client-side filters and sorting, then reveals three more items when an IntersectionObserver sentinel approaches the viewport. The PDF modal checks access status, requests private access when necessary, and fetches PDF bytes for display.

5. Active API route catalog

All paths below are prefixed with `/api/users` because the live server mounts the combined auth router at that prefix. Method names and paths are taken from `Backend/routes/auth.js`. Most administrative endpoints require an authenticated role-0 user even where the route middleware is written as `protect` followed by a role check.

Account and identity

- `POST /register` or `/SignUp`: create a visitor account and pending user request; send a verification code.
- `POST /login` or `/SignIn`: validate email/password, require verified local email, issue auth cookie.
- `POST /google/profile`: validate a Google credential and return profile details for the registration form.
- `POST /google`: create/find Google-backed account, optionally create a pending user request, and issue auth cookie.
- `GET /verify-email?token=...`; `POST /verify-email-code`; `POST /resend-verification`.
- `POST /forgot-password`; `POST /reset-password`; `GET /me`; `PUT /me`; `POST /logout`.

Researcher directory and publications

- `GET /researchers`: public profiles for enabled role-0/role-1 accounts plus approved publication summaries.
- `GET /publications`: approved, enabled publications, newest year first; strips private file-system and access-list fields.
- `POST /publications`: member/admin multipart submission with required PDF; new submissions start pending.
- `GET /publications/:id/pdf`: controlled PDF streaming; checks approval, account, author/admin or access-request grant, and increments downloads.
- `POST /publications/:id/access-request`; `GET /publications/:id/access-status`.
- `DELETE /publications/:id`: admin soft-delete by changing status to deleted.

Notifications and news

- `GET /notifications`: current user's newest notifications, limited to 50, with populated access-request context.
- `PATCH /notifications/:id/read`: mark only the current recipient's notification as read.
- `POST /publication-access-requests/:id/decision` with approved or rejected: decision by the responsible author; approval grants PDF access and notifies the requester.
- `GET /actualites`: approved news, newest first. `POST /actualites`: member/admin multipart news submission; status is pending.

Administrator routes

- GET /admin/stats; GET /admin/users; PUT /admin/users/:id; PUT /admin/users/:id/disable; PUT /admin/users/:id/restore.
- GET /admin/pending-users; POST /admin/pending-users/:id/approve; POST /admin/pending-users/:id/reject.
- GET /admin/publications; PUT /admin/publications/:id; PUT /admin/publications/:id/disable; PUT /admin/publications/:id/restore.
- GET /admin/pending-publications; POST /admin/pending-publications/:id/approve; POST /admin/pending-publications/:id/reject.
- GET /admin/actualites; GET /admin/pending-actualites; PUT /admin/actualites/:id; POST /admin/pending-actualites/:id/approve or /reject.

Do not confuse code presence with mounted routes

The newer split routers define /api/publications-style and /api/users-style route sets but are not imported by Backend/server.js. Frontend calls currently use /api/users/publications, /api/users/researchers and similar paths handled by auth.js.

6. Authentication, roles and access control

Account lifecycle

- Local signup normalizes email to lowercase, creates role 2 (visitor), stores a bcryptjs-hashed password, creates a PendingUserRequest, then emails a random six-digit code. The code hash is stored and expires after 15 minutes.
- Google ID tokens are verified server-side with the configured audience. Verified Google users can sign in without the local email-code step; new accounts are visitors and enter the pending-user workflow.
- Local sign-in rejects unverified accounts. Password reset tokens are random and opaque; only SHA-256 token hashes are stored, and reset expires in 60 minutes.
- A signed JWT contains user id, unique jti and auth-version number; expiry is 30 days. It is put in an HttpOnly, SameSite=Strict cookie, Secure when production or APP_URL uses HTTPS.
- protect reads the cookie, verifies JWT signature and expiry, checks revoked-token records, validates user id, rejects disabled accounts and compares authVersion. It attaches the database user to the request.
- Logout stores the token jti through its expiry and clears the cookie. A TTL index eventually removes expired revocation records. Password reset increments authVersion to invalidate older issued sessions.

Role map and protected actions

- Role 0 = administrator; role 1 = approved researcher/member; role 2 = visitor/default account.
- Public researcher listing includes enabled role-0 and role-1 users. Publication/news submission is limited to roles 0 and 1. Admin queues and management are role 0 only.
- Private PDFs are available to an author, administrator, explicitly granted user, or user with an approved access request. Non-private approved PDFs are public.

Abuse controls visible in the active router

- Authentication endpoints: 20 requests / 15 minutes per limiter key. Profile updates: 60 / 15 minutes; publication access requests: 30 / 15 minutes; uploads: 10 / 15 minutes.
- Reset/verification endpoints return generic messages in selected cases to reduce account-existence disclosure. Credentials and password-reset values are not included in this guide.

7. MongoDB entities and relationships

Collections and key fields

- User: identity/email/password hash, provider/Google ID, email verification/reset hashes and expiry times, authVersion, role/disabled flag, profile data, education, work history, languages, linked accounts, skills/research axes and denormalized publication summaries.
- Publication (collection explicitly named publications): title/type/year, abstract and article sections, journal/DOI/keywords/citation/download counts, authors with optional User references, submitter, status, PDF path, privacy/access users and publication-type detail fields.
- PendingUserRequest: one unique request per user with status pending/approved/rejected.
- PublicationAccessRequest: references a publication, requester and responsible author; unique index on publication + requester; status pending/approved/rejected.
- Notification: recipient, type, access-request reference, message and readAt timestamp.
- Actualite: news title/category/date/content/excerpt/image, submitter, important flag and moderation status.
- RevokedToken: unique jti plus expiry date, with MongoDB TTL index.

Relationships and consistency

Mongoose refs connect access requests to Publication and User records, and notifications to recipients and access requests. Publication authors can link to researcher accounts. On approval, publication summaries are copied into each linked User.publications array; updates remove the old summary and add a new one. This denormalizes data to simplify profile displays but creates synchronization work.

Important indexes include user role/disabled/creation time and verification/reset hashes; publication moderation status, year, submitter, author and privacy; actualite moderation/category/importance; access-request uniqueness and author/status; notification recipient/time; revoked-token TTL.

```
User (1) ----< PendingUserRequest
User (author/requester) ----< PublicationAccessRequest >---- Publication
User (recipient) ----< Notification >---- PublicationAccessRequest
Publication ----< authors[] (optional User links)
User.publications[] = denormalized summaries of approved Publication records
```

8. Algorithms and business workflows

Publication discovery

- The browser first filters by exact year, publication type and researcher (linked author or submitter). Keyword search is case-insensitive substring matching over title, abstract, journal, author names and keywords.
- Results sort by citations descending, title alphabetically, or year descending. The list initially shows 3 items; an IntersectionObserver with a 300 px root margin reveals the next 3.

Publication submit -> moderation -> profile sync

- Member/admin sends multipart metadata plus PDF. Server validates required title/type/year/file, normalizes strings, parses details/authors/keywords, and ensures the submitter is included as an author if not already linked.

- Publication record is created with pending status. Admin approval changes status to approved and syncs a compact publication summary to the submitter and linked researcher profiles; rejection leaves it out of public listing.
- Public list only returns approved and enabled records. PDF access is separately authorized; the PDF filesystem path is not included in list responses.

Private publication PDF request

- For an approved private publication, a signed-in non-author submits one access request. The unique publication/requester index prevents duplicate records; an existing rejected request can be returned to pending.
- The responsible author receives an in-app notification. On approval, the request status changes, requester id is added to pdfAccessUsers, and a decision notification is sent. The PDF authorization middleware also accepts author/admin identities and approved requests.

User and news moderation

- Signup creates visitor role and pending-user request. Admin approval promotes the account to role 1 (researcher); rejection changes request status.
- Member/admin-submitted news receives pending status. Admin may approve/reject it and edit fields. Public news API filters to approved records.

Password feedback and profile validation

- Signup's visual strength indicator awards points for length thresholds (6 and 10 characters), letters, numbers, and either special characters or mixed case; the score is feedback, not a substitute for server-side policy.
- Profile updates normalize strings, constrain item counts/lengths and cap summary word count in the modular user router. The live legacy profile normalizer has its own implementation; validation details can differ across the duplicated route files.

9. File handling and publication lifecycle

Storage and checks

- Multer disk storage writes PDF files under Backend/uploads using generated UUID names, maximum size 15 MB. News image files go under Backend/uploads/actualites with generated names and a 5 MB limit.
- Allowed image MIME types: JPEG/JPG, PNG, WebP and GIF. Server validates both MIME type and file signatures (PDF header, PNG magic bytes, JPEG marker, GIF signature, WebP RIFF/WEBP markers).
- PDF uploads are stored with a relative filesystem path in Publication.pdfPath. Public API returns hasPdf instead of the underlying path. News stores imageUrl as /uploads/actualites/<generated-name>.
- PDF route verifies the publication exists and is approved (except the coded admin path), checks private-access rules, verifies the physical file exists, increments downloadsCount and streams it.
- The server statically exposes only image extensions under /uploads/actualites; PDFs are not exposed through the static uploads route.

Durability and deployment implications

Uploads are local filesystem assets, not object storage. A multi-instance or ephemeral deployment must mount durable shared storage or migrate to object storage and update URLs/cleanup. Database backups alone do not preserve the uploaded files.

The modular fileUpload helper cleans up invalid signatures and has a reusable safeUnlink utility. The live legacy route contains separate upload handling, so confirm cleanup behavior and file retention against the mounted implementation before changing deployment assumptions.

10. Styling, UI libraries and assets

- Tailwind CSS 4 is imported through the Vite plugin. Frontend/src/index.css defines a light token palette (surface, text, primary, accent and error colors), theme extensions, base typography and page effects.
- Fonts include local Geist Variable package and CSS imports for Inter and Plus Jakarta Sans. Google Identity Services script is loaded in Frontend/index.html.
- Lucide icons are used throughout pages; Motion, Three.js and Recharts are declared for animation, canvas visuals and charts. The homepage uses ParticleCanvas; exact usage is component-specific.
- The component set includes Navbar, GoogleAuthButton, PublicationCard, PublicationsView, PdfViewerModal and expandable-card demos, plus ProfileReference sections and modal components.
- Public assets include EI&TIC logo, favicon and icons. Vite defines @ as an alias for Frontend/src.

Some generated or reference UI code may coexist with live page components. A dependency being declared does not imply that it is used on every route.

11. Configuration and local development

Environment variables (names only; no secret values)

- Backend .env.example: PORT, APP_URL, JWT_SECRET, MONGO_URI, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASSWORD, SMTP_FROM.
- Frontend/.env.example: VITE_GOOGLE_CLIENT_ID.
- APP_URL is used to build reset links and validate state-changing request origins; local default documented in the example is the Vite origin. Never commit real credentials or share .env contents.

Local startup

```
# From repository root: install root/server dependencies and frontend dependencies
npm install
npm --prefix Frontend install

# Configure local values in root .env and Frontend/.env (do not commit them)
npm run dev

# In another terminal, run the frontend dev server
npm --prefix Frontend run dev

# Production frontend bundle
npm run build
```

The backend starts on PORT (default 5000), connects to MONGO_URI before listening, and offers GET /health. Vite's development proxy expects the API at localhost:5000. A separate frontend production hosting/reverse-proxy arrangement is not defined by the root scripts; deploy the built Frontend/dist assets and route /api and /uploads to the backend.

12. Build, lint and deployment notes

Commands available in manifests

- npm run dev (root): start the Express server.
- npm run build (root): run Frontend build.

- `npm --prefix Frontend run dev`: Vite development server.
- `npm --prefix Frontend run build`: TypeScript build followed by Vite bundling.
- `npm --prefix Frontend run lint`: Oxlint.
- `npm --prefix Frontend run preview`: serve the built frontend for local preview.

Production considerations

- Set a strong private `JWT_SECRET`, production `MONGO_URI`, `HTTPS_APP_URL`, Google Identity client configuration and SMTP credentials; configure trusted reverse proxy and frontend routing.
- Ensure the application process can write Backend/uploads and that persistent storage/backups include this directory. Review upload/file lifecycle and retention.
- Use one canonical API route composition. Currently `legacy auth.js` duplicates behavior also found in modular route files, so future changes can diverge.
- Confirm browser cookie behavior, `APP_URL` origin checks, HTTPS Secure cookie setting, SMTP delivery, MongoDB indexes and admin seeding/role management in the target environment.
- No test script is declared in the visible package manifests. The build and lint scripts exist for the frontend; add/maintain automated API and browser tests as a separate engineering task.

13. Source tree map and runtime caveats

Main code locations

- Backend/server.js: Express bootstrap, security middleware, static images, health route, origin policy, database startup and active router mount.
- Backend/config/db.js: MongoDB connection. Backend/models/: User, Publication, Actualite, Notification, PendingUserRequest, PublicationAccessRequest and RevokedToken.
- Backend/routes/auth.js: live combined API router (authentication, profiles, admin moderation, publications, news, notifications and PDF access).
- Backend/routes/auth.routes.js, publication.routes.js, user.routes.js: split route implementations currently not composed by server.js.
- Backend/middleware/: authentication, publication-PDF authorization and shared error/ObjectId helpers. Backend/utis/: reusable auth helpers and upload validation.
- Frontend/src/App.tsx and main.tsx: app bootstrap and routing. Frontend/src/Pages/: route pages. Frontend/src/Components/: shared UI. Frontend/src/ProfileReference/: profile sections and modal components.
- Frontend/src/types.ts and data.ts: frontend data contracts and static content. Frontend/vite.config.ts, tsconfig*.json, components.json and .oxlintrc.json configure build/tooling.

Repository snapshot cautions

- The working tree contained many pre-existing modified, deleted and untracked files at documentation time. This guide describes files visible in the checkout and does not claim they are all committed or deployed.
- Both `Acceuil.tsx` and `Accueil.tsx` exist; `App.tsx` imports `Acceuil.tsx`. The duplicate should be treated as a separate/possibly stale page unless confirmed otherwise.
- Some pages/components are static or use fallback/demo data. The presence of a page, package, route module or model does not prove that it is wired into the live runtime.
- The repository root also contains temporary migration/test scripts. Their presence is not equivalent to a supported production migration command.

14. Operational observations and verification checklist

Read this before exposing the site publicly

- The active public researcher endpoints select contact fields such as email and phoneNumber. Confirm that exposing those fields publicly is intended; otherwise narrow the public projection.
- Active-user count is stored in a process-local Map and considers users seen in the last five minutes. It resets on restart and is not distributed across server instances.
- Backend/middleware/errorHandler.js exports shared 404/error handlers, but server.js does not currently attach them. Active routes handle many errors locally; verify unhandled route/error behavior.
- Admin dashboard and client route guards are UX controls, not security boundaries; server-side role checks are required and present on admin handlers. Review every new endpoint the same way.
- The auth cookie is HttpOnly, limiting JavaScript access, but review deployment proxy settings, HTTPS, CSRF/origin strategy and cookie behavior for the actual hosting topology.
- Signup creates user and pending request as separate database operations, with explicit rollback if pending request or verification email fails. Multi-document consistency could be strengthened with MongoDB transactions where supported.

Practical verification checklist

- 1. Set environment values locally without committing secrets; confirm MongoDB connectivity and GET /health.
- 2. Build frontend with npm run build and run npm --prefix Frontend run lint.
- 3. Exercise register -> email code -> sign in -> GET /me -> logout; test local and Google identity separately.
- 4. Test all three roles against profile edit, publication/news submission, admin queues and disabled-user behavior.
- 5. Submit valid and malformed PDF/image files; check size, signatures, public visibility, private request/approval/rejection and download count.
- 6. Verify file persistence, backup/restore, notification ownership, public researcher data projection and origin/cookie policy in the deployment environment.

Coverage statement

This is a source-based technical map, not an exhaustive line-by-line specification or a security certification. Endpoint behavior and production configuration should be revalidated whenever runtime wiring changes.